

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет «Школа информатики»

Иванова Арина Владимировна

**Система проверки доказательств алгебраических свойств
программ на функциональных языках**

Выпускная квалификационная работа - БАКАЛАВРСКАЯ РАБОТА
по направлению подготовки 01.03.02 Прикладная математика и информатика
образовательная программа «Прикладная математика и информатика»

Рецензент

кандидат ф.-м. н.

Вершилов А. В.

Руководитель

кандидат ф.-м. н., доцент

Москвин Д. Н.

Аннотация

В данной работе представлена система проверки цепочек эквивалентных преобразований на языке Haskell. Программы и структуры данных на языке Haskell обладают различными алгебраическими свойствами или, например, должны удовлетворять законы классов типов. Проверка данных свойств чаще всего осуществляется вручную или с использованием сторонних инструментов, основанных на SMT-решателях или языках с зависимыми типами. В отличие от них, данная система опирается на синтаксический анализ выражений, систему типов языка Haskell и оптимизационные механизмы компилятора. В работе реализовано разрешение α -эквивалентности, β и η -редукции, вычисление результатов работы функций и методов инстансов, а также механизмы замены паттернов на эквивалентные. Проведённые эксперименты показали, что система способна успешно подтверждать доказательства и обнаруживать ошибки в цепочках преобразований. Полученные результаты демонстрируют применимость предложенного подхода для проверки законов классов типов.

Ключевые слова: цепочки эквивалентных преобразований; синтаксический анализ; Haskell; GHC; законы классов типов.

Abstract

This work presents a program for verifying equational reasoning steps in Haskell programs, leveraging the language's type system, compiler substitution mechanisms, and syntactic analysis. Programs and data structures in Haskell often exhibit type class laws, which are traditionally verified manually or using external proof assistants or other tools, such as Liquid Haskell. They provide partial verification capabilities, but either do not support equational reasoning or rely on SMT solvers or strategy-based transformations, which can limit usability or clarity. In contrast, the proposed tool represents proofs as explicit chains of equivalences, checks type correctness using GHC, and validates each transformation syntactically against user-specified rules. The approach aims

to provide a lightweight, integrated solution for verifying type class laws for different instances.

Keywords: Equational reasoning; syntactic analysis; Haskell; GHC; Type Class Laws.

Содержание

Аннотация	2
Введение	6
1 Обзор литературы	9
1.1 Существующие аналоги	9
Выводы по главе	10
2 Технологии	12
2.1 Инструменты синтаксического анализа	12
Выводы и результаты по главе	14
3 Архитектурное устройство прогаммы	15
3.1 Базовые понятия	15
3.2 Разработка языка подсказок	16
3.3 Общая архитектура системы	19
3.4 Поиск первого различия выражений	19
3.5 Разрешение alpha-эквивалентности	20
3.6 Симплификатор (Simplifier)	21
3.7 Вычисления функций и постулатов	22
3.7.1 beta-редукция и eta-редукция	22
3.7.2 Вычисление определений функций	22
3.7.3 Вычисление методов инстансов	24
3.7.4 Вычисление постулатов	25
3.8 Ограничения	26
3.8.1 Рекурсивные функции	26
3.8.2 Сопоставление с образом, индукция и прочее	27
Выводы и результаты по главе	27
4 Эксперименты и оценка результатов	29
4.1 Пример использования	29

4.2 Эксперименты	30
4.2.1 Проверка корректных доказательств	30
4.2.2 Проверка некорректных доказательств	32
Выводы и результаты по главе	34
Заключение	35
Список источников	38

Введение

Программы и структуры данных промышленных функциональных языков, в частности Haskell, обладают различными алгебраическими свойствами. Поскольку библиотечные функции и компилятор опираются на корректность данных свойств, их соблюдение имеет ключевое значение для безошибочной работы многих программ.

Стандартным подходом к доказательству данных свойств является использование цепочек эквивалентных преобразований (equational reasoning). Это метод, основанный на последовательном преобразовании выражений с сохранением эквивалентности. Однако, проверка корректности подобных цепочек не может быть осуществлена тайпчекером или компилятором. Внутренние механизмы языка позволяют нам удостовериться лишь в совпадении типов, но не значений, в результате очередного преобразования.

Одним из способов является проверка вручную, что плохо подходит для сложных вычислений. Это может приводить к неточностям, особенно касательно деталей или контекста. Так, например, было показано, что стандартная реализация монады State не удовлетворяет законам монад в некоторых крайних случаях [1].

Другой способ проводить подобные вычисления – используя узкоспециализированные функциональные языки и системы доказательств теорем, например Agda или Coq, в которых используются полноценные зависимые типы. Их наличие позволяет контролировать доказательства с помощью тайпчекеров(систем проверки типов), которые фактически являются пруфчекерами(системами проверки доказательств). Однако подобные системы типов радикально усложняют программирование на этих языках, при том что благодаря достаточно гибкому синтаксису Haskell цепочки преобразования на нём же.

В то же время, доказательства таких свойств, как законы классов (к ним относятся законы монад или функторов) встречаются повсеместно. Так например, в исследовательской работе, направленной на создание еди-

ного способа для обхода деревьев [2] ключевую роль играли именно доказательства предложенных законов. Аналогичным образом, при разработке нового класса (аппликативного функтора) [3] возникают доказательства их законов.

Сравнение выражений на эквивалентность также является одной из основополагающих задач проверки программ, к которой можно подходить разными способами. Они могут включать тестирование подстановкой параметров, как QuickCheck, использование SMT-решателей, как в Liquid Haskell, или же сравнение выражений синтаксически и семантически. В основе последнего лежит сравнение абстрактных синтаксических деревьев (AST) и подстановка и вычисление функций. Заметим, что Glasgow Haskell Compiler (GHC) обладает широким спектром оптимизационных механизмов, включая упрощение и редуцирование функций и выражений. При этом система типов GHC автоматически обеспечивает проверку согласованности типов, что позволяет сосредоточиться на анализе корректности значений.

Принимая во внимание всё вышеперечисленное, целью данной работы является разработка и реализация системы для проверки доказательств теорем и алгебраических свойств программ на языке Haskell, выполненных в виде цепочек эквивалентных преобразований. При этом, максимально используя внутренние возможности компилятора, а финальную проверку осуществлять путём синтаксического анализа кода.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- 1) Разработать язык подсказок для обозначения выполняемых преобразований. Данная задача также включает выявление основных операций, используемых при доказательстве различных свойств и теорем.
- 2) Реализовать механизм сравнения AST (abstract syntax tree) для соседних выражений в цепочке, с учетом соответствующей подсказки
- 3) Изучить работу модуля оптимизации кода в GHC и с его помощью

реализовать подстановку функций и правил в анализируемые выражения.

- 4) Оценить работу полученного приложения на теоремах классов функторов и монад, а также законах аппликативного класса.

1 Обзор литературы

1.1. Существующие аналоги

Доказательство теорем – это фундаментальная задача, и, как уже упоминалось, существуют целые системы предназначенные для ее решение. Самые известные из них, Agda [4] и Coq, представляют собой собственные функциональные языки с зависимыми типами. Это концепция, вводящая значения переменных в определение типов, что позволяет использовать тайпчекер для проверки корректности доказательств и, как следствие, для верификации программ.

Однако, у такого подхода есть несколько существенных недостатков. Во-первых, алгебраические свойства можно доказать прямо в Haskell, написав на нём же цепочку преобразований. Для использования любой из представленных систем проверки доказательств потребуется трансформировать программу с Haskell и обратно, что приводит к вопросам эквивалентности уже таких трансформаций. Стоит отметить, что для Coq существует плагин `hs-to-coq` [5], созданный для решения данной проблемы. Во-вторых, подобные системы типов радикально усложняют программирование на этих языках и подразумевают владение теорией типов на уровне, более высоком, чем требуется для написания самого доказательства. Вследствие чего, использование Coq или Agda является избыточным для небольших программ.

Напротив, плагин Liquid Haskell [6] интегрирован в язык. Он позволяет пользователю указать логические свойства в аннотациях типов и автоматически проверить их, используя для этого SMT-решатель. И хотя Vazou и др. в своей работе [7] подробно изложили, как Liquid Haskell может быть использован для проверки цепочек преобразований, в нашей работе мы фокусируемся на возможностях компилятора GHC, а не на модификации стороннего приложения, которым является SMT решатель.

Одним из самых популярных инструментов тестирования в языке Haskell является QuickCheck [8]. Его основная идея заключается в основанном-

на-свойствах (property-based) тестировании, случайной генерации входных данных и поиске контрпримеров. Из данного описания видно, что QuickCheck отлично подходит для поиска ошибок, но не для формальной верификации поведения программы. Несмотря на это, в 2012 году группа исследователей [1] использовала его для создания фреймворка, подходящего для тестирования законов монад, в том числе с предоставленными цепочками преобразования.

Ещё одной значимой разработкой в данной области является плагин HERMIT GHC [9; 10]. Его последняя версия предоставляет возможность интерактивно работать с цепочками преобразований и автоматически гарантирует корректность выполняемых шагов. HERMIT — это система, интегрированная с GHC. Однако она ориентирована на преобразования с указанием конкретных стратегий, частью которых и являются цепочки преобразований. Это позволяет разработчикам использовать оптимизации компилятора, но при этом снижает наглядность и требует знания внутренней структуры GHC.

Выводы по главе

Существуют промышленные аналоги и научные работы, посвящённые доказательству теорем, проверке цепочек преобразований и доказательству эквивалентности выражений. Каждое из них обладает своим набором достоинств и недостатков.

В частности, функциональный язык Agda и система проверки доказательств Coq позволяют проверять сложные теоремы, но слабо подходят для регулярного использования в базовых ситуациях.

QuickCheck и Liquid Haskell написаны на Haskell и широки в применении, но в их основе лежат алгоритмы (случайная генерация, SMT-решатели), в то время как тех же результатов можно добиться, не выходя за рамки языка и компилятора.

И, наконец, практика работы с плагином HERMIT показала, что он также в полной мере не подходит для демонстрации цепочек эквивалент-

ных преобразований, хотя предлагает перспективную концепцию использования оптимизаций GNC, которой мы вдохновлялись.

Перечисленные недостатки существующих решений подчеркивают необходимость новых исследований и разработок в области проверки цепочек преобразований.

2 Технологии

2.1. Инструменты синтаксического анализа

В основе данного приложения лежат синтаксический и семантический анализ кода, поэтому одним из ключевых этапов работы был выбор AST представления, наилучшим образом подходящего для решения нашей задачи. Данное решение влияет на легкость последующих трансформаций, гарантии корректности, в частности типов, совместимость с различными библиотечными функциями, а также на семантическую точность, где важным вопросом является: возможно ли передать одно и то же действие используя разный синтаксис.

В рамках языка Haskell существует несколько альтернатив:

- Template Haskell [11]
- Сторонние библиотеки, например `haskell-src-ext` [12]
- GHC API – компилятор как библиотека – с AST для различных этапов компиляции: `Parsed`, `Renamed`, `Typechecked`
- GHC Core – следующая после `Typechecked` интерпретация программы

В ходе более подробного анализа каждого из представленных вариантов, были выявлены следующие преимущества и недостатки:

Template Haskell

Это расширение компилятора, позволяющее производить метавычисления во время компиляции. Оно хорошо подходит для конвертации кода в AST представление и обратно и для генерации кода, включая сериализацию и десериализацию. Полученный код также подвергается валидации на этапе компиляции и проходит проверку типов.

Вместе с тем, существенным минусом является многообразие синтаксических форм. Синонимы имен классов, сопоставление с образцами и отсутствие разрешения зависимостей имен для нас означают необходимость самостоятельно производить все вышеперечисленные преобразования и проверки, что является не только трудозатратно, но и повышает

вероятность ошибок.

Haskell-src-ext

Простая библиотека с понятным интерфейсом, однако одна из задач данной работы заключается в переиспользовании компилятора. В силу того что это самостоятельная библиотека, отсутствует совместимость с внутренними механизмами GHC.

GHC API

В своей сути, каждый из рассмотренных вариантов AST это репрезентация кода на разных стадиях компиляции. Они отвечают следующим трансформациям

- **Parsed** – сырое представление программы. Мало информативное но легко читаемое
- **Renamed** – с разрешением имен, что позволяет работать с уникальными идентификаторами
- **Typeschecked** – после проверки типов – это например позволяет проводить сравнение функций не только по идентификатору, но также по типу.

Отметим, что все три версии оперируют синтаксическим представлением практически полностью соответствующим исходному коду программы. Это делает код читаемым и простым для понимания, но влечет проблемы упомянутые при разборе Template Haskell.

GHC Core (desugar)

Это следующая интерпретация программы, полученная после `desugar` – то есть удаление синтаксического сахара и приведение к очень лаконичному виду. На данной стадии полученное AST наиболее близкое к лямбда-исчислению, и что не менее значимо, именно это представление сам компилятор Haskell использует для подстановок (`inlining`), `rewriting rules` и других оптимизаций. В результате чего, появляется возможность в полной мере воспользоваться его функционалом для выполнения наших задач поскольку они, при детальном рассмотрении, являются частными случаями

перечисленных оптимизаций. Принимая во внимание всё вышеперечисленное, мы остановили свой выбор именно на GHC Core представлении AST.

Выводы и результаты по главе

В этой главе мы рассмотрели несколько инструментов, предназначенных для получения абстрактного синтаксического дерева: библиотеку `haskell-src-ext`, расширение `Template Haskell` и библиотеку компилятора GHC API. Каждый из этих фреймворков расставляет различные приоритеты в вопросах соответствия исходному коду, семантической точности и совместимости с другими инструментами.

Так, например, сторонние библиотеки предоставляют удобный интерфейс использования, но не механизмы для подстановки и вычисления функций, которыми обладает оптимизатор GHC.

В то же время, AST представления близкие к исходному коду хорошо подходят для изменения стиля программы. Однако, они сохраняют значительную синтаксическую нагруженность, вызванную удобством использования, а не анализа языка.

Вместе с тем, компилятор Haskell имеет лаконичное семантическое представление, с уже выведенной информацией о типах, более того, он поддерживает оптимизации, связанные с редуцированием функций. По этим причинам мы считаем его наиболее подходящим инструментом для решения поставленных нами задач.

3 Архитектурное устройство прогаммы

3.1. Базовые понятия

Beta-редукция (β -reduction) – операция применения функции к аргументу, формально, может быть описана, как $(\lambda x.P)Q \rightarrow_{\beta} P[x := Q]$.

Eta-редукция (η -reduction) – операция упрощения функции, которая устраняет избыточное обёртывание, формульно $(\lambda x.fx) \rightarrow_{\eta} f$.

Абстрактное синтаксическое дерево (АСД, Abstract Syntax Tree, AST) – одна из форм промежуточного представления программ в виде древовидной структуры.

Альфа-эквивалентность – отношение эквивалентности между λ -выражениями, при котором они считаются одинаковыми, если они структурно эквивалентны, с возможными отличиями в именах связанных переменных.

Вычисление функции – получение результата применения функции к её аргументам.

Инлайнинг (Inlining, подстановка функции) – оптимизация, при которой вызов функции заменяется её телом.

Промежуточное представление (Intermediate Representation, IR) – структура данных или код, используемый внутри компилятора или виртуальной машины для представления программ.

Свободная переменная – переменная, не являющаяся аргументом лямбда-функции. В выражение: $(\lambda x.x + y)$ x - связанная переменная, а y - свободная переменная.

Связанная переменная – переменная, являющаяся аргументом лямбда-функции. И выражение: $(\lambda x.x + y)$ x - связанная переменная, а y - свободная переменная.

Связывание переменных (Binding) – механизм, определяющий область видимости и значение переменной внутри функции, формульно $(\lambda x.P)$.

3.2. Разработка языка подсказок

Ключевым оператором для проверки цепочек преобразований является `(==)`. Он позволяет связывать выражения между собой, при этом осуществляя проверку совпадения типов, а поскольку его результатом является второй переданный аргумент, данный оператор может быть использован для связи нескольких преобразований последовательно.

```
1 infixl 0 ==  
2 (==) :: a -> a -> a  
3 (==) _ y = y
```

Рис. 1.: Оператор связи выражений в цепочку преобразования

Кроме того, мы хотели иметь возможность указывать в нашей системе действие, соответствующее каждому переходу, поскольку в противном случае не всегда представляется возможным определить его однозначно. Всё что для этого потребуется, это определить еще один оператор `(--.)` принимающий выражение и подсказку, а возвращающий не измененное выражение:

```
1 (--.) :: a -> SideExprInfo -> a  
2 (--.) x _ = x
```

Рис. 2.: Оператор добавления подсказки

Здесь `SideExprInfo` это тип подсказки. Для того чтобы определить его значение, разберем на примерах какие объекты и действия встречаются при доказательстве теорем через цепочки преобразования.

Рассмотрим примеры, представленные на рисунке 3.

Стоит сразу заметить, что существуют функции трех разных видов. Во-первых, есть стандартные *определения* функций. Это те объекты, свойства которых мы хотим формулировать и доказывать. К примеру: `id`, `flip`, `(.)`. Их тела реализуют стандартную вычислительную функциональность.

На них в языке подсказок мы ссылаемся по имени, используя для упаковки конструктор

`Def :: String -> ExprInfo`, например `Def "id"` или `Def "(.)"`


```

1 theoremFunctor1 :: Monad m => m b -> m b
2 theoremFunctor1 m =
3     liftM id m                --- def liftM
4   === (m >>= return . id)    --- def (.)
5   === (m >>= \x -> return (id x)) --- def (.)
6   === (m >>= \x -> return x)  --- eta-reduction
7   === (m >>= return)         --- prop "m2"
8   === m                     --- QED
9
10 m2 :: Monad m => m a -> m a
11 m2 m = (m >>= return) === m    --- postulate
12
13 functor1LawMaybe Nothing =
14     fmap id Nothing          --- inst "fmap"
15   === Nothing               --- QED
16
17 betaRed f g x =
18     (\x' -> g (h x')) x    --- beta-reduction
19   === g (h x)

```

Рис. 3.: Примеры различных цепочек преобразования

Во-вторых, функции, для которых существует несколько реализаций, и выбор конкретного тела напрямую зависит от типов передаваемых параметров. Такие функции в Haskell называются *методами инстансов*.

В языке подсказок для них был добавлен конструктор

`Inst :: String -> ExprInfo`, например `Inst "fmap"`, `Inst "pure"`

В-третьих, есть специальные функции, тела которых представляют собой цепочки эквивалентных преобразований с подсказками. В нашем примере это `theoremFunctor1`. Эквивалентность начального и конечного подвыражений цепочки составляют содержание *высказывания*. Если же цепочка прошла процесс верификации, то мы называем высказывание *теоремой* и впоследствии, по имени функции можем сослаться на использование этой эквивалентности.

В языке подсказок для упаковки мы используем конструктор

`Prop :: String -> ExprInfo`, например `Prop "theoremFunctor1"`

Некоторые верные эквивалентности мы не можем вывести по тем или иным причинам. (Они могут задавать аксиоматику, например, законы классов типов; или же быть следствиями более общих математических теорий) Такие высказывания мы называем *постулатами* и маркируем под-

сказкой `Postulate`.

В довершение, преобразование может быть `beta` или `eta`-редукцией. Для них в языке подсказок используются соответствующие конструкторы

`Beta :: ExprInfo` и `Eta :: ExprInfo`

Также, каждое из действий может быть применено как напрямую, так и в обратную сторону. Или же, это можно интерпретировать, как применение действия напрямую к выражению слева или справа от оператора (`===`)

Изначально был использован встроенный в язык тип данных `Either`, который как раз имеет конструкторы `Left` и `Right`. Однако после, для упрощения работы и читаемости кода возникла потребность добавить новый вид подсказки `QED` и `Postulate`. Также стоит учитывать, что `Either` в контексте языка `Haskell` принято использовать для передачи верного (`Right`) и ошибочного значения (`Left`), а не напрямую как значения сторон. Так, например могла бы выглядеть безопасная функция деления.

```
1 divide :: Int -> Int -> Either String Int
2 divide _ 0 = Left "Exception: division by zero"
3 divide x y = Right (x div y)
```

Рис. 4.: Функция безопасного деления

Учитывая это, и что в нашем представлении скорее стандартным считается `Left` (применение функции к левой части выражения для получения правой), было принято решение добавить еще один собственный тип данных. Итоговый вариант представлен на рисунке 5

`data SideExprInfo :: L ExprInfo | R ExprInfo | QED | Postulate`

```
1 data SideExprInfo = L ExprInfo | R ExprInfo | QED | Postulate
2 data ExprInfo     = Def String | Prop String | Inst String | Beta | Eta
3
4 (---) :: a -> SideExprInfo -> a
5 (---) x _ = x
6
7 infixl 0 ===
8 (===) :: a -> a -> a
9 (===) _ y = y
```

Рис. 5.: Язык подсказок

3.3. Общая архитектура системы

Работа системы в целом состоит из двух отдельных этапов. Во-первых, сбор данных из полученного AST: выделение всех используемых функций (высказываний, определений и методов инстансов) и выделение всех переходов для каждого высказывания в лаконичную структуру, где `CoreExpr` это тип данных используемый в `Core` для конструирования выражений.

```
1 data Conversion = Conversion {  
2   cn_lhs  :: CoreExpr,  
3   cn_rhs  :: CoreExpr,  
4   cn_info :: SideExprInfo  
5 }
```

Рис. 6.: Тип данных для хранения информации о преобразовании

Во-вторых, проверка корректности для каждого перехода. В общем виде она состоит из последовательного поиска первого различия выражений, подстановки, beta-редукции и финального сравнения на alpha-эквивалентность. Каждый из этих этапов мы рассмотрим подробнее.

Заметим, что согласно нашему языку подсказок, постулаты являются частным случаем теорем. В связи с чем, становится особенно важна задача разрешения порядка обработки высказываний. С этой целью, перед началом анализа отдельных высказываний и переходов выполняется топологическая сортировка. Для этого используется модуль `Graph` со встроенным алгоритмом поиска сильно-связных компонент в графе и последующая топологическая сортировка. Нахождение такой нетривиальной компоненты означает наличие цикла в доказательствах, что приводит к завершению программы.

3.4. Поиск первого различия выражений

На данном этапе работы, мы имеем два выражения – до и после преобразования. Первое будем называть *исходным*, а второе *контрольным*. Следующая задача, которую решает наша программа – поиск в исходном выражении, такого минимального подвыражения, над которым предпола-

гается совершение преобразования из подсказки.

Альтернатива, выполнение действия над всем исходным выражением, однако это ведет к существенному минусу в производительности. При подстановке функции/паттерна во все возможные позиции, с контрольным выражением придется проделать ту же самую операцию, для корректности последующего сравнения, что означает в два раза больше работы.

Задача по нахождению такого подвыражения может решаться несколькими различными путями.

- поиском редекса – первого выражения которое можно выполнить. Существенным минусом такого подхода является то, что для применения функции должны быть переданы все аргументы, что сильно усложняет цепочки преобразований.
- поиском указанной функции/паттерна для постулатов и η -редукции. Для этого потребуется, или подстановка в каждую подходящую позицию поочередно, или указание пользователем последовательного номера примененной функции. В первом случае это ведет к увеличению числа сравнений между полученным выражением и контрольным. Во втором же, приводит к расчетам касательно порядка, ассоциативности и вложенности операций.
- Последовательное сравнение исходного и контрольного выражений до первого несовпадения. Далее эту операцию будем называть *поиском диффа*. Данный подход был выбран, поскольку он позволяет, по сравнению с поиском редекса, производить большее количество различных преобразований, а также не несет дополнительные вычислительные нагрузки. В его основе лежит семантическое сравнение и проверка на α -эквивалентность, для чего хорошо подходит используемое нами представление компилятора Core.

3.5. Разрешение α -эквивалентности

Мы сравниваем выражения по ходу программы в двух случаях: сравнение конкретных подвыражений при поиске диффа и итоговое сравнение.

Для сравнения на α -эквивалентность мы пользуемся предоставленным оператором сравнение (`==`) над типом данных `DeBruijn a = D CмEnv a`. Он является оберткой над типом `a` и, помимо значения, для каждого вхождения переменной хранит индекс, который отвечает за количество связываний между вхождением и объявлением. Это простая модификация позволяет избавиться от имён переменных и сравнивать выражения только семантически, что и требуется для α -эквивалентности.

В рамках конкретного выражения в лямбда исчислении любая переменная является либо связанной, либо свободной. Заметим, что при сравнении выражений целиком множества их свободных переменных совпадают. А вот при последовательном сравнении при поиске диффа мы самостоятельно снимаем связывания и теряем о них информацию. Поэтому потребовалось реализовать механизм по отслеживанию таких связываний. Для этого функция поиска диффа также имеет параметр (`cmEnv_e`, `cmEnv_ce`) который позволяет правильно сконструировать `DeBruijn`, аналогичный тому, что мы получили бы при сравнении выражений целиком.

3.6. Симплификатор (Simplifier)

Симплификатор [13], в оригинале – `simplifier`, это набор модулей компилятора GHC, которые в совокупности реализуют оптимизации в рамках семантического представления `Core`. Он несколько раз, последовательно применяет такие оптимизации как инлайнинг, `rewriting rules` (механизм замены одного паттерна на другой) и не только.

При этом, многие из ключевых функций и структур данных доступны для использования, что является неоспоримым плюсом. К ним относятся:

- `simplifyPrg` - предназначенная для упрощение программы целиком
- `simplifyExpr` - для упрощение выражения, в отрыве от окружающего контекста
- `simplExpr` - основная функция по оптимизации выражений, именно её вызывает `simplifyExpr` после инициализации пустыми значениями переменных окружения

- `substExpr` - функция для осуществления подстановки (инлайнинга)
- `lookupRule` - основная функция сопоставления левой части правила с выражением.

И многие другие. Всё это позволяет производить сложные вычисления и оптимизации, оставаясь, по большей части, уверенным в отсутствие ошибок или багов.

Стоит добавить, что одной из особенностей наиболее продвинутых функций, например `simplExpr`, является детальная связь с другими этапами компиляции и соответственно поддержание контекста. Из чего следует, что их использование не всегда оправдано, особенно в случае небольших выражений.

3.7. Вычисления функций и постулатов

3.7.1. beta-редукция и eta-редукция

Одним из преимуществ GHC Core является приведение программы к семантическому представлению. В ходе этой операции, которую также называют `desugar`, компилятор проводит некоторые оптимизации, в том числе, beta-редукцию. В связи с чем, эквивалентные с точностью до применения функции к аргументам, выражения в представлении Core будут alpha-эквивалентны.

Для разрешения eta-редукции достаточно найти дифф контрольного и исходного выражения, после чего, сравнить полученное подвыражение с образцом, соответствующим конструкции $(\lambda x \rightarrow f\ x)$.

3.7.2. Вычисление определений функций

Этот раздел посвящен тому, как в нашей системе обрабатываются переходы обозначенные подсказкой `Def "function-name"`.

Вычисление функции можно представить в два этапа – подстановку тела функции вместо ее имени (inlining) и подстановку аргументов в тело функции (beta-редукция). Очень удобно, что для осуществления обоих компилятор предоставляет функционал. Для инлайнинга – `substExpr`, со

следующей сигнатурой, где `Subst` это набор окружений, отвечающих за подстановку значений и типов.

```
substExpr :: Subst -> CoreExpr -> CoreExpr
```

В результате, мы получаем выражение, где все вхождения имени функции заменены на ее тело, что соответствует сразу нескольким преобразованиям. Однако при обработке цепочек преобразования мы подразумеваем одно действие на один переход, из-за чего возникает несоответствие между исходным и контрольным выражениями. Для разрешения данной ситуации нам как раз и понадобится поиск диффа. Его результат, в случае функции, это конструктор `App` (от *application* – применение) функции к её аргументам-типам.

Для осуществления *beta*-редукции используется функция

```
simpleOptExpr :: SimpleOpts -> CoreExpr -> CoreExpr
```

Которую мы слегка дорабатываем, чтобы избежать оптимизирующего инлайнинга. Отметим, что в последней версии GHC-14.1.1 есть функция `simpleOptExprNoInline` которую мы таким образом эмулируем.

Существуют альтернативы, описанные в части 3.6.

Но, хотя `SimplifyExpr` и `SimplExpr` выполняют необходимую задачу, они предусмотрены для последовательного упрощения, направленного на оптимизацию программы в целом. Дело в том, что вид выражения в доказательстве не всегда самый оптимальный, а скорее самый читаемый.

Также, потребовалось делать *beta*-редукцию пока она есть или не достигнута установленная верхняя граница, поскольку часть выражений выглядят как применение функций и пользователем воспринимаются соответственно, но на деле, являются лямбдами. Как в примере на рис. 7.

```

1      id . ($ x)                --. L (Def  ".")
2  === \f -> id (($ x) f)
3  --- equivalent ---
4      id . (\g -> g x)         --. L (Def  ".")
5  === \f -> id ((\g -> g x) f) --. L Beta
6  === \f -> id (\g -> f x)
```

Рис. 7.: Тип данных для хранения информации о преобразовании

3.7.3. Вычисление методов инстансов

В отличие от объектно-ориентированных языков программирования, в Haskell классы не являются объектами. Вместо этого, они представляют собой набор операций. Объект, в свою очередь, может являться инстансом – экземпляром класса и иметь методы - реализацию каждой операции из класса. Это способ добиться *ad hoc* полиморфизма и перегрузки функций в зависимости от типов параметров [14].

```
1 class Functor f where
2   fmap :: (a -> b) -> f a -> f b
3
4 instance Functor Maybe where
5   fmap _ Nothing = Nothing
6   fmap g (Just a) = Just (g a)
```

Рис. 8.: Пример класса Functor и его инстанса для типа Maybe

Для пользователя, метод инстанса ничем не отличается от обычной функции, однако, для его вычисления, идентификатора метода не достаточно, нужна информация о типах параметров. В связи с чем, для успешного выполнения нашей работы, был добавлен отдельный обработчик. Для его понимания необходимо разобраться в том, как этот процесс выполнен в компиляторе и какая информация уже хранится в Core и может быть нами использована.

Компилятору в процессе обработки программы необходимо, основываясь на типах переданных параметров, определить соответствующий инстанс и его методы. Данный процесс реализован на этапе проверки типов и проходит следующим образом:

- 1) На основе типов параметров разрешается инстанс, причем любой инстанс по своей сути это просто функция возвращающая все свои методы
- 2) Генерация селектора – функции, которая по инстансу возвращает конкретный метод.

Таким образом, например, обрабатывается вызов неизвестной функ-

ции `pure`

```
pure (Maybe a) (x :: Maybe a)
```

Становиться в `Core`:

```
pureSelector (Maybe a) $fMaybeA (Maybe a) (x :: Maybe a)
```

Где `cpure` - идентификатор, хранящий тело метода также как и любая простая функция, а `$fMaybeA` - инстанс.

Исходя из этого, вычисление метода инстанса превращается в задачу вычисления функции-селектора, вычисление инстанса и итоговое вычисление результата работы метода. Реализация каждого из этих этапов была разобрана в пункте 3.7.2

3.7.4. Вычисление постулатов

Как уже говорилось, в доказательстве может встретиться ситуация, когда в конкретном переходе хочется воспользоваться уже доказанным высказыванием – теоремой. Это делается с помощью подсказки `Prop ‘name’`

Проверка эквивалентности, в таком случае, сводится к двум пунктам. Во-первых, проверка - действительно ли в анализируемом переходе может быть применена указанная теорема, а во-вторых замена подвыражения на эквивалентное и итоговое сравнение с контрольным выражением.

Рассмотрим подробнее такой механизм оптимизации GHC как `rewriting rules`. Он позволяет пользователям указывать собственные правила трансформации, таким образом, сообщая компилятору, о возможности заменить один паттерн на другой.

Поскольку данный механизм выполняет очень близкую к нашей задачу, мы постарались его использовать для подстановки теорем.

Первоначально, использовалась предоставляемая компилятором функция `simplExpr`, которая, в том числе, выполняет `rewriting rules`. Но, как уже упоминалось, у такого подхода есть немало недостатков, связанных с соблюдением инвариантов контекста и форсированным применением правил.

Несмотря на все перечисленные сложности, использование `simplExpr`

для rewriting rules возможно. Мы предпочли ему более простой вариант `lookupRule` по двум причинам:

- `lookupRule` делает подстановку ровно 1 раз. В `simplExpr` нам приходилось делать запуск для обоих выражений – исходного и контрольного.
- `lookupRule` проще, и хоть и незначительно, но эффективнее `simplExpr`, поскольку вызывается в ходе исполнения `simplExpr` но не требует детального анализа окружения

Использование механизмов rewriting rules накладывает некоторые ограничения на то, какие высказывания могут быть преобразованы в Rules и, впоследствии использованы для доказательства других высказываний. Так, например, rules могут быть только выражения вида `f a1 a2 a3` – применение функции. Также, Rules накладывает ограничения на использование методов инстансов в правой части выражения, если их не было в левой. Причина за таким ограничением проста: селектор – рассмотренная ранее функция, сгенерированная компилятором для поиска методов инстанса, не является параметром для левой части правила, но появляется в правой.

3.8. Ограничения

3.8.1. Рекурсивные функции

Одним из ограничений нашей системы является обработка рекурсивных функций, а точнее, рекурсивных функций, в результате подстановки которых, в головной позиции оказывается снова та же функция.

```
foldl f z (x:xs) = foldl f (f z x) xs
```

Проблема заключается в том, что при сравнении, мы никак не можем получить разницу в самой функции, а только в ее аргументах, что не позволяет отследить применение функции. В рамках продолжения работы над данным проектом рассматривается возможность добавление отдельной подсказки и механизма обработки для таких функций.

Данное исключение не касается ситуаций, в которых после применения функции на первую позицию выходит та же, но она не рекурсивная, в

следующем примере:

```
const const 2 3 9 => const 3 9 => 3
```

Поскольку на самом деле, первый и второй `const` отличаются по типам

```
const_outer :: (a -> b -> a) -> a -> (a -> b -> a)
```

```
const_inner :: a -> b -> a
```

Заметим, что для нерекурсивных функций, это будет верно всегда, поскольку `f_inner` является результатом работы `f_outer`, но не встречается в определении `f_outer` (функция не рекурсивна), и мы делаем ровно одно преобразование, следовательно `f_inner` приходит из параметров. А значит, тип `f_inner` вложен в тип `f_outer` и они не могут совпадать.

3.8.2. Сопоставление с образом, индукция и прочее

При разработке системы на неё был наложен ряд ограничений, включая сопоставление с образцом и обработку рекурсивных типов данных, требующая математической индукции.

Кроме того, система ориентирована на последовательный анализ цепочек преобразований и однозначное определение выполняемого действия на каждом шаге. В связи с чем, рассматривались только преобразования отвечающие еденичному действию.

Не смотря на всё вышеперечисленное, возможно применение система для проверки доказательств ряда утверждений, что показали проведенные нами эксперименты.

Выводы и результаты по главе

В данной главе были рассмотрены архитектурные и алгоритмические основы разработанной системы. Были введены понятия, необходимые для понимания данной работы. Также мы подробно рассмотрели сценарии использования и на их основе разработали язык подсказок. После чего была проведена оценка алгоритмов предназначенных для поиска различий между выражениями.

Были подробно рассмотрены механизмы оптимизации компилятора

Haskell и предоставляемый ими функционал, который, впоследствии, мы использовали для реализации β и η -редукции, а также вычисления определений функций, методов инстансов и постулатов.

В завершение главы, мы перечислили основные ограничения накладываемые на нашу систему. В результате чего, было получено детальное представление об архитектуре данного проекта.

4 Эксперименты и оценка результатов

4.1. Пример использования

Для использования нашей системы, потребуется переписать доказательства высказываний согласно разработанному нами языку подсказок, а также указать все используемые постулаты и функции. После чего файл передается на запуск программы. В результате работы, формируется отчет, состоящий из перечисления подтвержденных в корректности теорем и высказываний, в которых система обнаружила ошибки или столкнулась с не реализованным функционалом, вместе с кратким обоснованием ошибок. На рис.9 представлен пример высказывания с подсказками, а на рис.10 отчета.

```
1 app4LawArrow :: (e -> b -> c) -> (e -> a -> b) -> (e -> a) -> e -> c
2 app4LawArrow h g u =
3   pure (.) <*> h <*> g <*> u                --. L (Inst "<*>")
4 === (\x -> pure (.) x (h x)) <*> g <*> u      --. L (Inst "pure")
5 === (\x -> (.) (h x)) <*> g <*> u            --. L (Inst "<*>")
6 === (\y -> (\x -> (.) (h x)) y (g y)) <*> u  --. L Beta
7 === (\y -> (.) (h y) (g y)) <*> u           --. L (Inst "<*>")
8 === (\z -> (\y -> (.) (h y) (g y)) z (u z)) --. L Beta
9 === (\z -> (.) (h z) (g z) (u z))           --. L (Def ".")
10 === (\z -> h z (g z (u z)))                --. R (Inst "<*>")
11 === (\z -> h z ((g <*> u) z))              --. R (Inst "<*>")
12 === h <*> (g <*> u)                          --. QED
```

Рис. 9.: Основной пример использования. Высказывания с подсказками

```
===== REPORT =====
Successfully proved: 5
  app4LawArrow
  app3LawArrow
  app2LawArrow
  app0LawArrow
  app0LawCmps

Failures proved: 4
  app0LawCmps
  app2LawCmps
  app3LawCmps
  app4LawCmps
Reasons for failures:

Fail in decl: app0LawCmps in conversion number: 3
Reason:
Rule `["ID: a0"]`not fired.
```

Рис. 10.: Пример части отчета

4.2. Эксперименты

Для проведения качественной оценки разработанной системы было составлено два эксперимента. В первом эксперименте оценивалась способность системы подтверждать корректные доказательства, а во втором — отвергать некорректные. Все эксперименты проводились в среде Haskell (GHC 9.12.2), под управлением cabal 3.14.2.0, на MacOS 15.7.4.

4.2.1. Проверка корректных доказательств

Для проведения эксперимента был составлен датасет, включающий несколько видов высказываний.

Во-первых, несколько простых утверждений о свойствах композиции (TH-triv): ассоциативность, композиция с нейтральным элементом (`id`) и `flip . flip === id`.

Во-вторых, доказательство законов классов типов для конкретных инстансов класса.

1) (TH-std) Законы классов типов для стандартных типов данных.

- Functor для Maybe, Either, Arrow (`->`), Pair (`,`), List `[]`, State
- Applicative для Maybe, Either, Arrow (`->`)
- Monad для Maybe, Either, Arrow (`->`)

2) (TH-Cmps) Законы классов Functor и Applicative для пользовательского типа данных

```
newtype Cmps f g x = Cmps (f (g x))
```

В-третьих, утверждений связанных с законами классов типов. Утверждений о связи между законами внутри класса, например выводимость второго закона функтора из первого (TH-связи). Проверка законности реализации методов базового класса, через методы дочернего (TH-derived).

Последнее возникает, поскольку Haskell поддерживает расширение классов, задавая между ними отношение родитель-наследник. При этом, класс наследник обязан подчиняться как своим законам, так и родительским. К примеру, класс Applicative является наследником класса Functor,

поэтому задает функцию `liftA`, которая соответствует методу `fmap` класса `Functor`. Мы проверяем, что `liftA` удовлетворяет законам класса `Functor`.

Помимо этого, часть классов типов может быть задана разным набором методов. Так для класса `Monad` существует альтернатива, со своим набором законов

```

1 class Applicative m => Monad m where
2   (>>=) :: m a -> (a -> m b) -> m b
3   return :: a -> m a
4
5 class Functor m => AltMonad m where
6   ret :: a -> m a
7   joi :: m (m a) -> m a

```

Рис. 11.: Определение классов `Monad` и `AltMonad`

Однако, такие классы являются взаимозаменяемыми, следовательно, используя законы одного класса, законы второго могут быть доказаны. Мы рассматривали 15 теорем для пары `Monad` – `AltMonad` и 28 теорем класса `Monoidal`, который является заменой `Applicative`.

В ходе эксперимента было проанализировано 102 высказывания и обработано 28 постулатов. Анализ высказывания считался успешным если система завершила работу и не обнаружила ошибок в цепочке преобразований. В результате были получены следующие данные.

Группа высказываний	Успех	Не успех	Всего
TH-triv	5	-	5
TH-std Functor	9	3	12
TH-std Applic.	15	-	15
TH-std Monad	9	-	9
Cmps Functor	2	-	2
Cmps Applic.	1	4	5
TH-связи	2	-	2
TH-derived	9	-	9
AltMonad	15	-	15
Monoidal	-	28	28
Итого	67	35	102

Таблица 1.: Результаты эксперимента о проверке корректности доказательств

Как мы видим, система во всех случаях завершилась в штатном режиме, однако все полученные ошибки связаны с ограничениями системы.

Законы класса `Functor` для списка не получится доказать без наличия аппарата математической индукции, а для доказательства первого закона для `State` требуется анализ внутренней структуры типа, что ведет к сопоставлению с образом, которое также не реализовано в нашей системе.

Реализация инстанса класса `Applicative` для типа `Cmps` содержит сравнение переменных разных, но эквивалентных типов, что является ограничением системы. При этом, законы `Functor`, хоть и содержат конструкции приведения одного типа к другому, не требуют их сравнения.

Основа класса `Monoidal` – это операции над парами и тройками, что ведет к неявному сравнению с образцом.

Результат данного эксперимента демонстрируют, что разработанная система успешно справляется с проверкой законов классов типов для классических типов данных. Также корректно обрабатываются утверждения относящиеся к классам типов в общем случае, как например теоремы эквивалентности классов `Monad` и `AltMonad`.

4.2.2. Проверка некорректных доказательств

Второй эксперимент включал в себя три высказывания, содержащие разные категории ошибок. Ожидаемым результатом являлось завершение с классификацией результата как “не подтверждено” либо “ошибка выполнения” с указанием причины отказа.

В первом, проверялось возможность системы отслеживать циклические зависимости между высказываниями, в результате чего система аварийно завершилась с соответствующим сообщением об ошибке, как показано на рисунке 12


```

app1Law_cyrcle :: Applicative f => f a -> f a
app1Law_cyrcle as =
    pure id <*> as    --. L (Prop "a0_cyrcle")
=== fmap id as      --. L (Prop "f1")
=== id as           --. L (Def "id")
=== as              --. QED
Error. Cycle dependencies were found.
[[app1Law_cyrcle, a0_cyrcle]]

a0_cyrcle :: Applicative f => f a -> f a
a0_cyrcle as =
    pure id <*> as    --. L (Prop "app1Law_cyrcle")
=== as               --. R (Inst "fmap")
=== fmap id as       --. QED

```

Рис. 12.: Пример высказывания и результата - ошибка цикл

Второе высказывание, содержало некорректный переход, а точнее пропущенное преобразование. Поскольку наша система рассматривает каждое преобразование как единичную операцию над выражением, то в результате обработки мы получаем не эквивалентные выражения, о чем система нам сообщает в отчете об анализе данного высказывания. Последнее вы-

```

1 fish2_skipStep :: AltMonad m => (a -> m b) -> a -> m b
2 fish2_skipStep k =
3     (k >=>.. ret)      --. L (Def ">=>..")
4 --- (joi . fmap ret . k) --. L (Prop "composeAssoc")
5 === ((joi . fmap ret) . k) --. L (Prop "mjfr2")
6 === (id . k)           --. L (Prop "composeLeftNeutral")
7 === k                  --. QED

```

```

===== REPORT =====
Successfully proved: 0
| <None>
Failures proved: 1
| fish2_skipStep
Reasons for failures:
Fail in decl: fish2_skipStep in conversion number: 1
Reason:
Error. Expr (1) after conversion not equal expr (2)
Expected: Application :: . @(m_a15b b_a15d)
Got: Application :: . @(m_a15b (m_a15b b_a15d))

```

Рис. 13.: Пример высказывания и результата - ошибка не эквивалентны

сказывание содержало ошибку в несоответствие подсказки и выполняемого действия, в результате чего, система не смогла совершить подстановку указанной в подсказке функции и справедливо посчитала данную цепочку некорректной.

Полученные данные, иллюстрируют способность системы отклонять некорректные доказательства, что свидетельствует о надежности предложенного подхода при обработке заведомо ошибочных цепочек преобразо-

ваний.

Выводы и результаты по главе

В данной главе было приведено несколько примеров использования разработанной системы. Также были описаны два эксперимента необходимые для качественной оценки. Первый эксперимент был направлен на оценку способности системы подтверждать корректность доказательств, тогда как второй — на оценку способности системы обнаруживать и отклонять некорректные цепочки преобразований.

Полученные результаты показали, что система способна корректно обрабатывать высказывания, доказывающие свойства типов классов `Functor`, `Applicative` и `Monad` для большинства, из рассмотренных, классических структур данных. Обнаруженные ошибки явно продемонстрировали ограничения системы в анализе индукции и сопоставления с образцом.

В ходе второго эксперимента были проанализированы случаи неуспешного завершения программы и возникновения ошибок выполнения, что также свидетельствует о надежности системы.

Заключение

В рамках данной работы была реализована система для проверки цепочек эквивалентных преобразований на языке Haskell. Подобная система применима для проверки доказательств различных теорем и алгебраических свойств, что особенно актуально для проверки соблюдения законов классов типов. В ходе работы были рассмотрены существующие аналоги, включающие узкоспециализированные системы проверки доказательств Agda и Coq, а также плагины HERMIT и Liqued Haskell.

В нашей работе, мы сфокусировались на возможностях компилятора, позволяющих проверять выражения на совпадения типов. Это позволило нам разработать язык подсказок для обозначения выполняемых в доказательстве переходов.

Проведя анализ существующих технологий и библиотек для синтаксического анализа мы остановили свой выбор на представлении компилятора Core, благодаря этому, нам удалось реализовать алгоритм сравнения выражений и представления их в удобной для последующего анализа форме.

Помимо этого, мы рассмотрели возможности оптимизатора GHC - симплификатора, что позволило реализовать подстановку и вычисление определений, методов инстантов и постулатов.

Для оценки работоспособности разработанной системы была проведена серия экспериментов на наборах корректных и некорректных доказательств. Полученные результаты показали, что разработанная система способна успешно верифицировать корректные цепочки преобразований и обнаруживать ошибки в некорректных. Высказывания представленные в экспериментах сфокусированы на различных аспектах законов классов типов `Functor`, `Applicative` и `Monad`, включая их доказательство для конкретных представителей класса, доказательство соблюдения отношений родитель-наследник, а также проверке теорем эквивалентности классов `Monad` и `AltMonad`. Большинство из перечисленных высказываний были подтверждены нашей

системой, что свидетельствует о её успешной реализации.

Также были выявлены основные ограничения, определяющие дальнейшее направление развития:

- Доработка и модификация существующего решения для возможности обработки сопоставления с образцом.
- Разработка модуля математической индукции. Заметим, что в функциональных языках индукция, в основном, строится на рекурсивных типах данных, которые в свою очередь связаны с сопоставлением с образцом. По этой причине решение первого пункта является наиболее актуальным.

Список источников

1. *Jeuring J., Jansson P., Amaral C.* Testing type class laws // Proceedings of the 2012 Haskell Symposium. — 2012. — с. 49–60.
2. Understanding idiomatic traversals backwards and forwards / R. Bird [и др.] // Proceedings of the 2013 ACM SIGPLAN Symposium on Haskell. — 2013. — с. 25–36.
3. *Capriotti P., Kaposi A.* Free Applicative Functors // Proceedings 5th Workshop on Mathematically Structured Functional Programming. — 2014. — с. 2–30.
4. *Mu S.-C., Ko H.-S., Jansson P.* Algebra of Programming in Agda: Dependent Types for Relational Program Derivation // journal of Functional Programming. — 2009. — т. 19, вып. 5. — с. 545–579.
5. Total Haskell is reasonable Coq / A. Spector-Zabusky [и др.] // Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs. — 2018. — с. 14–27.
6. *Vazou N.* Liquid Haskell: Haskell as a Theorem Prover // Dissertation, uni. of California, San Diego. — 2016.
7. Theorem Proving for All: Equational Reasoning in Liquid Haskell (Functional Pearl) / N. Vazou [и др.] // In Proceedings of the 11th ACM SIGPLAN International Symposium on Haskell (Haskell 2018). — 2018. — с. 132–144.
8. *Claessen K., Hughes J.* QuickCheck: a lightweight tool for random testing of Haskell programs // Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming. — 2000. — с. 268–279.
9. *Farmer A.* HERMIT: Mechanized Reasoning during Compilation in the Glasgow Haskell Compiler // PhD thesis, University of Kansas. — 2015.
10. *Farmer A., Sculthorpe N., Gill A.* Reasoning with the HERMIT: tool support for equational reasoning on GHC core programs. // In Proceedings of the 2015 ACM SIGPLAN Symposium on Haskell (Haskell '15). — 2015. — т. 50, вып. 12. — с. 23–34.
11. template-haskell [электронный ресурс] : Template Haskell documentation. — URL: https://downloads.haskell.org/ghc/latest/docs/users_guide/xts/template_haskell.html (дата обр. 18.05.2026).
12. haskell-src-ext [электронный ресурс] : haskell-src-ext library package. — URL: <https://hackage.haskell.org/package/haskell-src-xts> (дата обр. 18.05.2026).
13. *Jones S., Marlow S.* Secrets of the Glasgow Haskell Compiler inliner. // journal of Functional Programming. — 1999. — т. 12, вып. 5. — с. 394–434.

14. Type classes and overloading [электронный ресурс] : Type classes and overloading. — URL: <https://www.haskell.org/tutorial/classes.html> (дата обр. 18.05.2026).